2. The frontend engineer runs a test against the mock provider at unit test time, and the mock provider checks if the actual request from the consumer is registered as an interaction. If so, the mock provider returns the expected response, and the consumer confirms that the expected response is understood.
3. If all tests of this type are successful, a contract will be generated automatically, which includes the defined interaction. The contract will then be uploaded to a shared broker that the actual data provider can access. There are different ways to store the contract, which we'll cover as part of chapter 9.

We'll further explain the process of writing a consumer contract test in chapter 4.

1.4.2 Contract testing from the provider side

On the provider side, which will typically be found in a separate project repository, one of the backend engineers has been notified that a new contract was generated by the consumer. As part of the contract testing on the provider side, the provider verifies the contract. Similar to the consumer test, the provider doesn't communicate with the actual consumer but with a mock consumer instead. A provider contract test, illustrated in figure 1.6, can be explained on a high level in a few steps:

1. The provider downloads the contract generated by the consumer from the shared broker.
2. Using the contract testing tool, the expected request is replayed, and the provider sends the actual response to the expected request.
3. The contract testing tool compares the actual response and the expected response defined by the consumer in the contract. If the results match, the verification is successful.